

REGISTRO DE BANCO DE DADOS - SEMANA 16

Etapas:

1. Limpeza do banco

- a) **id_cliente** é a chave primária da tabela clientes.
- b) **id_cliente** da tabela pedidos aponta para clientes.
- c) Para garantir que a **quantidade seja maior que zero**, evitando valores inválidos, como 0 ou números negativos.

2. Tabela clientes

Cria o cadastro dos clientes, armazenando **ID, nome e e-mail**. O ID identifica cada cliente.

```
INSERT INTO clientes (nome, email) VALUES
('Ana Souza', 'ana@fasttech.com'),
('Bruno Lima', 'bruno@fasttech.com'),
('Carla Mendes', 'carla@fasttech.com');

INSERT INTO produtos (nome, categoria, preco) VALUES
('Notebook', 'Informatica', 3500.00),
('Mouse', 'Informatica', 120.00),
('Cadeira Gamer', 'Moveis', 950.00),
('Monitor 24', 'Informatica', 900.00);

INSERT INTO pedidos (data_pedido, id_cliente, status) VALUES
('2026-08-10', 1, 'Pago'),
('2026-08-10', 2, 'Pendente'),
('2026-08-11', 3, 'Pago');

INSERT INTO itens_pedido (id_pedido, id_produto, quantidade) VALUES
(1, 1, 1),
(1, 2, 2),
(2, 4, 1),
(3, 3, 1);

SELECT * FROM clientes;
SELECT * FROM produtos;
SELECT * FROM pedidos;
SELECT * FROM itens_pedido;
```

3. Tabela produtos

Quantas linhas foram exibidas?

4 linhas.

Por que o pedido 1 pode aparecer mais de uma vez?

Porque o **pedido 1 possui mais de um produto**. Cada produto do pedido gera uma linha diferente no resultado do JOIN.

4. Tabela pedidos

Registra os pedidos feitos pelos clientes. O id_cliente liga cada pedido ao seu cliente.

```
CREATE INDEX idx_produtos_categoria
ON produtos(categoria);

CREATE INDEX idx_produtos_categoria_preco
ON produtos(categoria, preco);

SHOW INDEX FROM produtos;

EXPLAIN
SELECT *
FROM produtos
WHERE categoria = 'Informatica'
AND preco > 1000;
```

Evidência 1:

```

Output
-----
id_cliente | nome          | email              |
-----
1 | Ana Souza    | ana@fasttech.com |
2 | Bruno Lima   | bruno@fasttech.com |
3 | Carla Mendes | carla@fasttech.com |
-----

id_produto | nome          | categoria | preco |
-----
1 | Notebook     | Informatica | 3500.00 |
2 | Mouse        | Informatica | 120.00 |
3 | Cadeira Gamer | Moveis      | 950.00 |
4 | Monitor 24   | Informatica | 900.00 |
-----

id_pedido | data_pedido | id_cliente | status |
-----
1 | 2026-08-10 | 1 | Pago |
2 | 2026-08-10 | 2 | Pendente |
3 | 2026-08-11 | 3 | Pago |
-----

id_item | id_pedido | id_produto | quantidade |
-----
1 | 1 | 1 | 1 |
2 | 1 | 2 | 2 |
3 | 2 | 4 | 1 |
4 | 3 | 3 | 1 |
-----

id_pedido | cliente | produto | quantidade |
-----
1 | Ana Souza | Notebook | 1 |
1 | Ana Souza | Mouse | 2 |
2 | Bruno Lima | Monitor 24 | 1 |
3 | Carla Mendes | Cadeira Gamer | 1 |
-----

Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expr |
-----
produtos | 0 | PRIMARY | 1 | id_produto | A | 4 | NULL | NULL |  | BTREE |  |  | YES | NULL |
produtos | 1 | idx_produtos_categoria | 1 | categoria | A | 2 | NULL | NULL |  | BTREE |  |  | YES | NULL |
produtos | 1 | idx_produtos_categoria_preco | 1 | categoria | A | 2 | NULL | NULL |  | BTREE |  |  | YES | NULL |
produtos | 1 | idx_produtos_categoria_preco | 2 | preco | A | 4 | NULL | NULL |  | BTREE |  |  | YES | NULL |
-----

id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
-----
1 | SIMPLE | produtos | NULL | range | idx_produtos_categoria,idx_produtos_categoria_preco | idx_produtos_categoria_preco | 407 | NULL | 1 | 100.00 | Using index condition |
-----

pedidos_antes |
-----
3 |
-----

id_pedido | data_pedido | id_cliente | status |
-----
1 | 2026-08-10 | 1 | Pago |
2 | 2026-08-10 | 2 | Pendente |
3 | 2026-08-11 | 3 | Pago |
4 | 2026-08-13 | 3 | Pendente |
-----

pedidos_depois |
-----
3 |
-----

```

id_pedido	data_pedido	id_cliente	status
1	2025-08-10	1	Pago
2	2025-08-10	2	Pendente
3	2025-08-11	3	Pago
5	2025-08-14	2	Pago

id_pedido	cliente	produto	categoria	quantidade	preco	total_item	status
1	Ana Souza	Notebook	Informatica	1	3500.00	3500.00	Pago
1	Ana Souza	Mouse	Informatica	2	120.00	240.00	Pago
2	Bruno Lima	Monitor 24	Informatica	1	900.00	900.00	Pendente
3	Carla Mendes	Cadeira Gamer	Moveis	1	950.00	950.00	Pago

id_pedido	cliente	produto	categoria	quantidade	preco	total_item	status
1	Ana Souza	Notebook	Informatica	1	3500.00	3500.00	Pago
3	Carla Mendes	Cadeira Gamer	Moveis	1	950.00	950.00	Pago
2	Bruno Lima	Monitor 24	Informatica	1	900.00	900.00	Pendente
1	Ana Souza	Mouse	Informatica	2	120.00	240.00	Pago

id_pedido	cliente	produto	categoria	quantidade	preco	total_item	status
1	Ana Souza	Notebook	Informatica	1	3500.00	3500.00	Pago
1	Ana Souza	Mouse	Informatica	2	120.00	240.00	Pago
2	Bruno Lima	Monitor 24	Informatica	1	900.00	900.00	Pendente
3	Carla Mendes	Cadeira Gamer	Moveis	1	950.00	950.00	Pago

total_clientes	total_produtos	total_pedidos	total_itens	total_registros_view	total_registros_materializada
3	4	4	4	4	4

Explique brevemente o que este print comprova:

O print comprova que as tabelas do banco de dados foram criadas e que os dados foram inseridos e consultados corretamente no OneCompiler, mostrando que a estrutura e os relacionamentos do banco estão funcionando conforme o esperado.

Evidência 2: Resultado do JOIN

Explique brevemente o que este print comprova:

O print comprova que o JOIN conseguiu relacionar corretamente as quatro tabelas, mostrando o pedido, cliente, produto e quantidade em uma única consulta.

```
SELECT
|   p.id_pedido,
|   c.nome AS cliente,
|   pr.nome AS produto,
|   i.quantidade
FROM pedidos AS p
INNER JOIN clientes AS c
|   ON p.id_cliente = c.id_cliente
INNER JOIN itens_pedido AS i
|   ON p.id_pedido = i.id_pedido
INNER JOIN produtos AS pr
|   ON i.id_produto = pr.id_produto
ORDER BY p.id_pedido, i.id_item;
```

EVIDÊNCIA 3 — Índices / EXPLAIN QUERY PLAN

Explique brevemente o que este print comprova:

O print comprova que os **índices foram criados corretamente** e que o banco consegue utilizá-los para **otimizar e acelerar as consultas** realizadas na tabela produtos.

```
EXPLAIN
SELECT *
FROM produtos
WHERE categoria = 'Informatica'
|   AND preco > 1000;
```

AULA 3 — Parte 2: transações e views

Etapa 5:

Antes da transação havia 3 pedidos.

Após o ROLLBACK ficaram 3 pedidos.

O pedido temporário permaneceu salvo? (X) Não

Etapa 6:

Explique a diferença observada entre COMMIT e ROLLBACK:

O **COMMIT** confirma e salva definitivamente as alterações realizadas na transação. Já o **ROLLBACK** desfaz as alterações, fazendo o banco voltar ao estado anterior à transação.

Etapa 7:

Resposta:

A vantagem é que a **VIEW já possui os JOINS e cálculos prontos**, então podemos consultar os dados de forma mais simples, sem precisar escrever novamente todo o código. Isso facilita o uso e evita repetição.

Etapa 8:

Essa etapa cria uma tabela com o resultado da consulta da relatorio_vendas, contendo apenas os produtos com preço maior que 100.

Ela simula uma view materializada, pois o resultado fica armazenado fisicamente na tabela. Se os dados originais mudarem, a tabela precisa ser recriada para atualizar as informações.

EVIDÊNCIA 4 — ROLLBACK ou COMMIT

id_pedido	data_pedido	id_cliente	status
1	2025-08-10	1	Pago
2	2025-08-10	2	Pendente
3	2025-08-11	3	Pago
5	2025-08-14	2	Pago

id_pedido	cliente	produto	categoria	quantidade	preco	total_item	status
1	Ana Souza	Notebook	Informatica	1	3500.00	3500.00	Pago
1	Ana Souza	Mouse	Informatica	2	120.00	240.00	Pago
2	Bruno Lima	Monitor 24	Informatica	1	900.00	900.00	Pendente
3	Carla Mendes	Cadeira Gamer	Movelis	1	950.00	950.00	Pago

id_pedido	cliente	produto	categoria	quantidade	preco	total_item	status
1	Ana Souza	Notebook	Informatica	1	3500.00	3500.00	Pago
3	Carla Mendes	Cadeira Gamer	Movelis	1	950.00	950.00	Pago
2	Bruno Lima	Monitor 24	Informatica	1	900.00	900.00	Pendente
1	Ana Souza	Mouse	Informatica	2	120.00	240.00	Pago

id_pedido	cliente	produto	categoria	quantidade	preco	total_item	status
1	Ana Souza	Notebook	Informatica	1	3500.00	3500.00	Pago
1	Ana Souza	Mouse	Informatica	2	120.00	240.00	Pago
2	Bruno Lima	Monitor 24	Informatica	1	900.00	900.00	Pendente
3	Carla Mendes	Cadeira Gamer	Movelis	1	950.00	950.00	Pago

total_clientes	total_produtos	total_pedidos	total_itens	total_registros_view	total_registros_materializada
3	4	4	4	4	4

ROLLBACK;

SELECT COUNT(*) AS pedidos_depois
FROM pedidos;

Explique brevemente o que este print comprova:

O print comprova que a transação foi executada corretamente, mostrando o resultado da alteração no banco após o uso de COMMIT ou ROLLBACK.

EVIDÊNCIA 5 — VIEW relatorio_vendas

```
SELECT COUNT(*) AS pedidos_antes
FROM pedidos;

START TRANSACTION;

INSERT INTO pedidos (data_pedido, id_cliente, status)
VALUES ('2026-08-13', 3, 'Pendente');

SELECT * FROM pedidos;

ROLLBACK;

SELECT COUNT(*) AS pedidos_depois
FROM pedidos;
```

O print comprova que a VIEW relatorio_vendas foi criada e está funcionando corretamente, reunindo informações de clientes, pedidos e produtos em uma única consulta e mostrando o total de cada item.

9. Questões finais

1. Qual é a função de uma chave primária?

Identificar cada registro de forma única.

2. Qual é a função de uma chave estrangeira?

Relacionar uma tabela com outra.

3. Por que um índice pode melhorar uma consulta?

Porque deixa a busca pelos dados mais rápida.

4. Qual é a diferença entre COMMIT e ROLLBACK?

COMMIT salva a alteração; ROLLBACK desfaz.

5. Para que serve uma VIEW?

Para facilitar consultas já prontas.

6. Qual é a diferença entre a view comum e a simulação de view materializada?

A **VIEW comum** consulta os dados na hora; a **materializada simulada** armazena o resultado em uma tabela.